

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

Khoa Công nghệ thông tin 1



Đề tài: Phát triển hệ thống đặt phòng khám trực tuyến sử dụng kiến trúc microservice

Giảng viên:	Kim Ngọc Bách
Nhóm lớp:	03
Nhóm thực hiện:	21
Sinh viên thực hiện:	Mã SV
Đinh Công Thắng	B22DCCN808
Phạm Hoàng Anh	B22DCCN039

Hà nội, 2026

MỤC LỤC

MỤC LỤC.....	1
1. MÔ TẢ BÀI TOÁN.....	2
1.1. Vấn đề thực trạng.....	2
1.2. Giải pháp đề xuất.....	2
1.3. Mục tiêu của hệ thống.....	2
2. KIẾN TRÚC HỆ THỐNG.....	3
2.1. Tổng quan về kiến trúc và tính chất của hệ thống.....	3
2.2. Các thành phần lõi và vai trò của Middleware.....	3
2.3. Cơ chế giao tiếp liên tiến trình (Inter-process Communication).....	4
2.4. Tính mở và khả năng mở rộng (Openness & Scalability).....	4
3. MÔ TẢ CÁC MICROSERVICE.....	5
3.1. User Service (Cổng 8081).....	5
3.2. Doctor Service (Cổng 8082).....	5
3.3. Appointment Service (Cổng 8083).....	5
3.4. Notification Service (Cổng 8084).....	6
4. THIẾT KẾ RESTful API.....	6
4.1. Nguyên lý thiết kế giao diện dịch vụ.....	6
4.2. Cấu trúc định tuyến và định dạng dữ liệu.....	7
4.3. Đặc tả các giao diện lập trình ứng dụng.....	7
5. LƯỒNG GIAO TIẾP HÀNG ĐỢI THÔNG ĐIỆP.....	8
5.1. Cơ sở lý thuyết và vai trò của giao tiếp bất đồng bộ.....	8
5.2. Kiến trúc và các thành phần tham gia.....	8
5.3. Chi tiết luồng xử lý sự kiện đặt lịch.....	9
5.4. Đánh giá tính ưu việt của thiết kế hàng đợi.....	9
6. THỬ NGHIỆM VÀ ĐÁNH GIÁ.....	10
6.1. Môi trường và cấu hình triển khai.....	10
6.2. Kịch bản thử nghiệm luồng nghiệp vụ.....	10
6.3. Đánh giá ưu điểm kiến trúc.....	14
6.4. Hạn chế và hướng phát triển.....	15

1. MÔ TẢ BÀI TOÁN

1.1. Vấn đề thực trạng

Hiện tại, công tác quản lý và sắp xếp lịch khám của bác sĩ tại các cơ sở y tế và bệnh viện vẫn còn gặp nhiều khó khăn do phụ thuộc vào các quy trình thủ công. Bệnh nhân thường phải đến tận nơi để ghi danh trực tiếp, dẫn đến tình trạng quá tải và mất nhiều thời gian chờ đợi. Bên cạnh đó, hệ thống hiện tại thiếu cơ chế thông báo tự động, khiến bệnh nhân không được cập nhật kịp thời về lịch hẹn của mình, dễ dẫn đến tình trạng quên lịch hoặc nhầm lẫn thông tin.

1.2. Giải pháp đề xuất

Để giải quyết những bất cập trên, dự án đề xuất xây dựng một Hệ thống đặt lịch khám bệnh trực tuyến ứng dụng kiến trúc Microservice hiện đại. Hệ thống được thiết kế nhằm cung cấp các tiện ích toàn diện cho các nhóm đối tượng:

- Cho phép người dùng dễ dàng đăng ký tài khoản, xem danh sách thông tin bác sĩ và chủ động đặt lịch khám trực tuyến mọi lúc, mọi nơi.
- Cung cấp công cụ để quản lý danh sách bác sĩ, thiết lập lịch khám và phân bổ bệnh nhân một cách khoa học, hiệu quả.
- Hệ thống tích hợp tính năng tự động gửi thông báo xác nhận lịch hẹn cho bệnh nhân ngay sau khi thao tác đặt lịch được thực hiện thành công.

1.3. Mục tiêu của hệ thống

Việc phát triển và đưa hệ thống vào ứng dụng thực tế hướng tới các mục tiêu cốt lõi sau:

- Tự động hóa các quy trình, qua đó giảm thiểu khối lượng công việc quản lý lịch khám thủ công cho nhân viên y tế.
- Cải thiện trải nghiệm khám chữa bệnh của bệnh nhân, mang lại sự tiện lợi và tiết kiệm thời gian.

- Tăng hiệu quả quản lý và sử dụng quỹ thời gian làm việc của đội ngũ bác sĩ.
- Ứng dụng kiến trúc Microservice giúp hệ thống dễ dàng bảo trì và hỗ trợ mở rộng quy mô khi số lượng người dùng và dịch vụ của bệnh viện tăng lên trong tương lai.

2. KIẾN TRÚC HỆ THỐNG

2.1. Tổng quan về kiến trúc và tính chất của hệ thống

Theo định nghĩa chuẩn, hệ thống phân tán là một tập hợp các phần tử điện toán tự trị hoạt động độc lập nhưng xuất hiện trước người dùng như một hệ thống gắn kết duy nhất,. Để đạt được điều này, "Hệ thống đặt lịch khám bệnh" đã từ bỏ cách tiếp cận thiết kế khối đơn khối (monolithic) truyền thống - vốn dĩ gộp chung mọi thành phần thành một chương trình khổng lồ rất khó thay thế hay nâng cấp,. Thay vào đó, hệ thống ứng dụng kiến trúc Microservice, chia nhỏ ứng dụng thành các dịch vụ mang tính tự trị cao, qua đó phân rã độ phức tạp và cho phép mở rộng quy mô một cách linh hoạt.

Về mặt kiến trúc tổng thể, hệ thống là sự kết hợp của kiến trúc lai ghép (Hybrid Architecture),. Nó vừa chứa các phần tử mang tính tập trung để xử lý định tuyến ban đầu (API Gateway), vừa chứa các phần tử phân tán (Các Microservice) hoạt động theo mô hình khách/chủ (Client-Server) và cả kiến trúc xuất bản - đăng ký (Publish-Subscribe),.

2.2. Các thành phần lõi và vai trò của Middleware

Hệ thống được tổ chức thành các tầng phần mềm với các thành phần chính như sau:

- **Tầng Middleware (API Gateway & RabbitMQ):** Để giúp các ứng dụng phân tán giao tiếp dễ dàng, hệ thống sử dụng một lớp phần mềm trung gian gọi là Middleware, nằm giữa hệ điều hành và các ứng dụng,.

- **API Gateway:** Đóng vai trò như một **Broker** trung tâm, chuyên tiếp nhận yêu cầu từ máy khách và định tuyến đến đúng ứng dụng cần thiết. Cơ chế này giúp che giấu sự phân tán của các tiến trình, mang lại **tính trong suốt (Distribution Transparency)**, khiến end-user không cần biết chính xác dịch vụ nào đang chạy ở đâu,..
- **Message Queue (RabbitMQ):** Đóng vai trò là một kênh sự kiện (Event Channel) ở tầng Middleware, giúp các tiến trình giao tiếp mà không bị phong tỏa.
- **Tầng Ứng dụng (Các Microservice):** Bao gồm 4 dịch vụ cốt lõi (User, Doctor, Appointment, Notification). Bằng cách đặt các thành phần logic khác nhau trên các tiến trình/máy chủ khác nhau, hệ thống thực hiện sự phân tán theo chiều dọc (vertical distribution).
- **Kho dữ liệu phân tán (Distributed Datastores):** Dữ liệu không lưu trữ tập trung mà được phân rã cho từng dịch vụ quản lý độc lập (PostgreSQL cho User, Doctor, Appointment) nhằm tăng hiệu năng, tuân thủ nguyên tắc độc lập dữ liệu và phục vụ giao tiếp theo yêu cầu bài toán,..

2.3. Cơ chế giao tiếp liên tiến trình (Inter-process Communication)

Giao tiếp là thành phần sống còn của bất kỳ hệ thống phân tán nào. Hệ thống này sử dụng hai mô hình giao tiếp song song để giải quyết yêu cầu bài toán:

Được xây dựng dựa trên mô hình Khách/Chủ (Client/Server) cơ bản với hành vi yêu cầu - trả lời (request-reply). Khi Client gửi một yêu cầu (ví dụ: lấy danh sách bác sĩ), tiến trình của Client sẽ gọi qua REST API và phải chờ đợi kết quả trả về từ Server. Phương pháp này phù hợp cho các tương tác yêu cầu phản hồi tức thời để đảm bảo tính tuần tự của quy trình.

Hệ thống áp dụng kiến trúc Publish-Subscribe (Xuất bản - Đăng ký). Cụ thể hơn, đây là mô hình kênh sự kiện (Event Channel Model), trong đó các tiến trình trao đổi thông tin thông qua sự lan tỏa sự kiện trên một kênh trung gian. Khi Appointment Service tạo lịch khám thành công, nó đóng vai trò là Publisher đẩy một sự kiện lên kênh. Notification Service đóng vai trò Subscriber, đã đăng

ký trước, sẽ nhận sự kiện này để xử lý. Ưu điểm cốt lõi của mô hình này là tạo ra sự ràng buộc lỏng (Loose Coupling) giữa các tiến trình; các thành phần không cần tham chiếu trực tiếp đến nhau và bên gửi có thể tiếp tục công việc ngay sau khi đẩy thông điệp lên hàng đợi mà không bị phong tỏa (non-blocking),,.

2.4. Tính mở và khả năng mở rộng (Openness & Scalability)

Thông qua việc sử dụng RESTful API với định dạng chuẩn JSON và Message Queue, hệ thống tuân thủ mục tiêu thiết kế về tính mở. Các giao diện được định nghĩa rõ ràng cho phép các thành phần từ các nhà phát triển khác nhau, hoặc viết bằng ngôn ngữ khác nhau, dễ dàng tích hợp và làm việc cùng nhau thông qua chuẩn chung,. Đồng thời, bằng cách tách biệt chức năng thành các Microservice và sử dụng giao tiếp bất đồng bộ để che giấu trễ truyền thông, hệ thống đạt được mục tiêu về khả năng mở rộng (Scalability), sẵn sàng đáp ứng khi quy mô dữ liệu và người dùng tăng lên,. Toàn bộ được đóng gói qua Docker/Docker-compose để dễ dàng triển khai độc lập.

3. MÔ TẢ CÁC MICROSERVICE

Trong kiến trúc hệ thống phân tán, việc phân rã hệ thống thành các vi dịch vụ (Microservice) đòi hỏi phải tuân thủ nghiêm ngặt nguyên lý thiết kế phân định giới hạn ngữ cảnh (Bounded Context). Mỗi dịch vụ trong "Hệ thống đặt lịch khám bệnh" hoạt động như một tiến trình tự trị, tập trung giải quyết một chức năng nghiệp vụ cụ thể và tương tác với nhau để tạo thành một hệ thống gắn kết. Để đảm bảo tính độc lập dữ liệu và tránh điểm lỗi cục bộ, mỗi vi dịch vụ lỗi đều quản lý một kho dữ liệu phân tán riêng biệt. Cụ thể, hệ thống được cấu thành từ các dịch vụ sau:

3.1. User Service (Cổng 8081)

- **Phạm vi nghiệp vụ:** Chịu trách nhiệm quản lý vòng đời người dùng, bao gồm đăng ký, đăng nhập và bảo mật thông tin.

- **Đặc tính hệ thống:** Dịch vụ lưu trữ dữ liệu tại cơ sở dữ liệu PostgreSQL mang tên `user_db`. Điểm nổi bật về mặt kiến trúc là việc áp dụng cơ chế xác thực bằng JSON Web Token. Đây là hình mẫu lý tưởng của việc thực thi phi trạng thái trong hệ thống phân tán. Quá trình này cho phép máy chủ không cần lưu giữ trạng thái phiên của máy khách sau khi hoàn tất yêu cầu, qua đó giúp dịch vụ giảm thiểu tiêu hao bộ nhớ và dễ dàng mở rộng quy mô theo chiều ngang khi số lượng bệnh nhân gia tăng.

3.2. Doctor Service (Cổng 8082)

- **Phạm vi nghiệp vụ:** Quản lý toàn bộ danh mục y tế, bao gồm thông tin chi tiết của các chuyên khoa, hồ sơ bác sĩ và lịch làm việc.
- **Đặc tính hệ thống:** Dữ liệu được cô lập hoàn toàn tại `doctor_db` với dữ liệu tham chiếu gốc về các chuyên khoa. Trong mô hình giao tiếp đồng bộ, Doctor Service đóng vai trò là một tiến trình máy chủ luôn ở trạng thái lắng nghe trên cổng 8082, sẵn sàng cung cấp dữ liệu tham chiếu thông qua các RESTful API cho các dịch vụ khác truy vấn tính hợp lệ của lịch khám.

3.3. Appointment Service (Cổng 8083)

- **Phạm vi nghiệp vụ:** Đây là miền nghiệp vụ lõi, chịu trách nhiệm điều phối toàn bộ quy trình và các trạng thái xác nhận hay hủy bỏ của lịch khám bệnh. Dịch vụ này sở hữu kho dữ liệu riêng là `appointment_db`.
- **Giao tiếp liên tiến trình:** Dịch vụ này thể hiện sự tương tác đa hình thái. Thứ nhất, nó đóng vai trò là máy khách khi gọi thủ tục đồng bộ sang Doctor Service để kiểm tra dữ liệu. Thứ hai, sau khi giao tác lưu lịch khám vào cơ sở dữ liệu thành công, nó chuyển sang vai trò nhà xuất bản trong mô hình kiến trúc hướng sự kiện bằng cách sinh ra một thông điệp tạo lịch hẹn mới và đẩy vào nút mạng chuyên mạch của hệ thống hàng đợi RabbitMQ.

3.4. Notification Service (Cổng 8084)

- **Phạm vi nghiệp vụ:** Cung cấp dịch vụ chạy nền nhằm gửi thông báo xác nhận lịch hẹn cho bệnh nhân.
- **Đặc tính hệ thống:** Hoàn toàn không sử dụng cơ sở dữ liệu để lưu trữ vĩnh viễn, Notification Service là một tiến trình phi trạng thái. Trong mô hình kênh sự kiện, nó vận hành dưới vai trò người tiêu thụ. Ngay khi hệ thống khởi động, tiến trình này thiết lập liên kết với RabbitMQ và đăng ký lắng nghe trên hàng đợi sự kiện. Việc tách rời logic gửi thông báo ra khỏi tiến trình đặt lịch giúp triệt tiêu sự phong tỏa tiến trình, tạo ra một sự ràng buộc lỏng hoàn hảo nhằm đảm bảo hiệu năng và thời gian đáp ứng tức thời cho ứng dụng máy khách.

4. THIẾT KẾ RESTful API

4.1. Nguyên lý thiết kế giao diện dịch vụ

Để các tiến trình trong hệ thống phân tán có thể trao đổi thông tin và tương tác với nhau theo cơ chế đồng bộ, hệ thống áp dụng kiến trúc chuyển giao trạng thái đại diện RESTful. Theo cách tiếp cận này, hệ thống được xem như một tập hợp các tài nguyên phân tán, mỗi tài nguyên được quản lý bởi một vi dịch vụ và được định danh duy nhất thông qua một đường dẫn định vị tài nguyên thống nhất.

Giao tiếp giữa máy khách và máy chủ tuân thủ chặt chẽ mô hình yêu cầu và trả lời thông qua giao thức HTTP. Đặc tính quan trọng nhất của thiết kế RESTful trong hệ thống này là sự thực thi phi trạng thái. Sau khi máy chủ hoàn tất việc xử lý một yêu cầu và trả về kết quả, nó sẽ không lưu giữ bất kỳ thông tin nào về trạng thái của máy khách, điều này giúp hệ thống tiết kiệm tài nguyên bộ nhớ và dễ dàng nhân bản máy chủ để mở rộng quy mô.

4.2. Cấu trúc định tuyến và định dạng dữ liệu

Toàn bộ các giao tiếp đồng bộ từ phía máy khách đều đi qua cổng giao tiếp trung tâm API Gateway tại địa chỉ cơ sở là `http://localhost:8080`. Dữ liệu trao đổi giữa các tiến trình được định dạng theo chuẩn JSON nhằm đảm bảo tính mở và khả năng tích hợp độc lập với mọi nền tảng phần cứng hay ngôn ngữ lập trình. Hệ thống cung cấp một giao diện đồng nhất dựa trên các phương thức tiêu chuẩn: thao tác GET dùng để truy vấn trạng thái tài nguyên, POST dùng để tạo mới, PUT dùng để cập nhật và DELETE dùng để xóa tài nguyên.

4.3. Đặc tả các giao diện lập trình ứng dụng

- **Giao diện dịch vụ người dùng:** Tập trung quản lý vòng đời và xác thực người dùng. Điểm truy cập POST tại đường dẫn `/users/register` cho phép tạo mới tài khoản và POST tại `/users/login` dùng để cấp phát mã thông báo xác thực. Các thao tác quản trị khác bao gồm truy vấn danh sách toàn bộ người dùng bằng phương thức GET tại `/users/`, hoặc thao tác với từng cá nhân cụ thể qua các lệnh GET, PUT và DELETE tại đường dẫn `/users/:id`.
- **Giao diện dịch vụ bác sĩ:** Đóng vai trò cung cấp khối dữ liệu tham chiếu y tế cho toàn hệ thống. Quản trị viên sử dụng lệnh POST tại `/doctors/` để thêm mới hồ sơ bác sĩ. Để tra cứu thông tin, máy khách gọi lệnh GET tại `/doctors/` để lấy danh sách tổng hợp, hoặc GET tại `/doctors/:id` để trích xuất hồ sơ chi tiết kèm theo lịch làm việc thực tế của một bác sĩ. Các lệnh PUT và DELETE tại `/doctors/:id` được cấp quyền cho việc cập nhật hoặc xóa hồ sơ.
- **Giao diện dịch vụ đặt lịch:** Đây là nhóm giao diện phức tạp nhất vì nó chi phối miền nghiệp vụ lõi của hệ thống. Quá trình tạo mới một lịch khám được thực hiện qua lệnh POST tại đường dẫn `/appointments/`. Hệ thống thiết kế các đường dẫn tra cứu chéo rất linh hoạt nhằm đáp ứng tính đa hình trong truy xuất dữ liệu: máy khách có thể gọi GET tại `/appointments/patient/:patientId` để liệt kê toàn bộ lịch khám của một bệnh nhân, hoặc gọi GET tại `/appointments/doctor/:doctorId` để trích xuất lịch làm việc theo góc nhìn của bác sĩ. Việc chuyển đổi trạng thái của quy

trình khám bệnh như xác nhận hay hủy bỏ được thực hiện thông qua lệnh PUT tại `/appointments/:id/status`.

- **Giao diện dịch vụ thông báo:** Mặc dù hoạt động cốt lõi của dịch vụ thông báo dựa trên cơ chế bất sự kiện bất đồng bộ ở chế độ nền, dịch vụ này vẫn cung cấp một giao diện đồng bộ thông qua lệnh GET tại đường dẫn `/notifications/`. Giao diện này cho phép máy khách hoặc hệ thống quản trị truy vấn lại danh sách và kiểm tra trạng thái của các thông báo đã được xử lý.

5. LUỒNG GIAO TIẾP HÀNG ĐỢI THÔNG ĐIỆNP

5.1. Cơ sở lý thuyết và vai trò của giao tiếp bất đồng bộ

Trong các hệ thống phân tán, việc chỉ sử dụng giao tiếp đồng bộ thông qua kiến trúc chuyển giao trạng thái đại diện tiềm ẩn rủi ro về mặt hiệu năng. Nếu dịch vụ đặt lịch phải chờ dịch vụ thông báo gửi xong email hoặc tin nhắn cho bệnh nhân rồi mới phản hồi lại cho máy khách, tiến trình của máy khách sẽ bị phong tỏa và dẫn đến lãng phí tài nguyên mạng.

Để giải quyết vấn đề này, hệ thống áp dụng phương pháp truyền thông bền bỉ thông qua phần mềm trung gian hướng thông điệp. Hệ thống hàng đợi thông điệp hỗ trợ đặc lực truyền thông không đồng bộ, không bắt buộc các tiến trình đầu cuối phải đồng thời ở trạng thái hoạt động. Phương pháp này mang lại sự ràng buộc lỏng giữa các vi dịch vụ, giúp hệ thống nâng cao thông lượng và triệt tiêu hiện tượng thất cổ chai.

5.2. Kiến trúc và các thành phần tham gia

Hệ thống sử dụng RabbitMQ làm lớp phần mềm trung gian quản lý hàng đợi thông điệp. Luồng giao tiếp được thiết kế theo mô hình kênh sự kiện với nền tảng là kiến trúc xuất bản và đăng ký, trong đó:

- **Kênh sự kiện trung gian:** Là hàng đợi mang tên `appointment.created.queue` được cấu hình trên RabbitMQ.

- **Tiến trình xuất bản sự kiện:** Dịch vụ đặt lịch đóng vai trò là nhà xuất bản. Khi có một giao tác mới hoàn thành, nó đóng gói thông tin thành một thông điệp và đẩy lên hàng đợi.
- **Tiến trình tiêu thụ sự kiện:** Dịch vụ thông báo đóng vai trò là người đăng ký. Ngay khi hệ thống khởi động, nó đã thiết lập liên kết đến RabbitMQ và liên tục lắng nghe sự kiện trên hàng đợi này.

5.3. Chi tiết luồng xử lý sự kiện đặt lịch

Quy trình giao tiếp bất đồng bộ thông qua hàng đợi được diễn ra theo các bước tuần tự như sau:

- **Bước 1:** Tiến trình máy khách gửi yêu cầu đặt lịch khám đến điểm truy cập của dịch vụ đặt lịch.
- **Bước 2:** Dịch vụ đặt lịch thực hiện các thao tác xác thực logic đồng bộ. Nếu hợp lệ, nó tiến hành lưu thông tin lịch khám mới với trạng thái xác nhận vào cơ sở dữ liệu.
- **Bước 3:** Ngay sau khi giao tác lưu trữ thành công, dịch vụ đặt lịch sinh ra một thông điệp thông báo sự kiện tạo lịch khám và phát hành lên kênh truyền.
- **Bước 4:** Thông điệp được hệ thống RabbitMQ tiếp nhận và đưa vào hàng đợi `appointment.created.queue` để lưu trữ an toàn. Ngay tại thời điểm này, dịch vụ đặt lịch coi như đã hoàn thành nhiệm vụ và lập tức trả về kết quả thành công cho người dùng mà không bị phong tỏa tiến trình.
- **Bước 5:** Dịch vụ thông báo đang lắng nghe trên hàng đợi sẽ lập tức bắt được thông điệp, tiến hành bóc tách dữ liệu và thực hiện các tác vụ chạy nền để gửi thông báo đến bệnh nhân một cách độc lập.

5.4. Đánh giá tính ưu việt của thiết kế hàng đợi

Dịch vụ thông báo hoạt động hoàn toàn độc lập và không làm gián đoạn luồng công việc chính của dịch vụ đặt lịch. Nếu dịch vụ thông báo gặp sự cố và sụp đổ, hệ thống đặt lịch khám vẫn hoạt động bình thường để phục vụ bệnh nhân.

Các thông điệp được lưu trữ bền bỉ trên hàng đợi của RabbitMQ. Ngay cả khi bên nhận không hoạt động, thông điệp vẫn nằm an toàn trong hàng đợi cho đến khi dịch vụ thông báo khởi động lại và tiêu thụ chúng, đảm bảo không có thông báo nào bị thất lạc.

Khi số lượng bệnh nhân tăng cao dẫn đến lượng thông điệp tạo ra lớn, người quản trị chỉ cần khởi tạo thêm nhiều phiên bản của dịch vụ thông báo để cùng lắng nghe trên một hàng đợi, hệ thống sẽ tự động phân tải công việc mà không cần phải thay đổi bất kỳ dòng mã lập trình nào.

6. THỬ NGHIỆM VÀ ĐÁNH GIÁ

6.1. Môi trường và cấu hình triển khai

Để đảm bảo tính nhất quán giữa môi trường phát triển và môi trường thực thi, hệ thống áp dụng kỹ thuật ảo hóa mức hệ điều hành thông qua công nghệ đóng gói Docker và công cụ điều phối Docker Compose. Môi trường thử nghiệm yêu cầu tối thiểu nền tảng Node.js phiên bản 20, hệ quản trị cơ sở dữ liệu PostgreSQL phiên bản 16 và hệ thống quản lý thông điệp RabbitMQ phiên bản có giao diện quản trị. Việc khởi chạy toàn bộ hệ thống phân tán bao gồm cổng giao tiếp API Gateway và bốn vi dịch vụ độc lập được thực hiện tự động chỉ qua một tập lệnh cấu hình duy nhất. Cách tiếp cận này giúp che giấu sự phức tạp của quá trình cài đặt, mang lại tính trong suốt về mặt phân bố cho người quản trị.

6.2. Kịch bản thử nghiệm luồng nghiệp vụ

Cách Chạy Hệ thống

Bước 1: Clone repository

```
git clone https://github.com/DCThang293/hospital_appointment_system.git
```

```
cd hospital_appointment_system
```

Bước 2: Chạy tất cả service bằng Docker Compose

```
docker compose up -d
```

Bước 3: Kiểm tra các service đã chạy

```
# Xem danh sách container
```

```
docker compose ps
```

```
# Xem log từ tất cả services
```

```
docker compose logs -f
```

Bước 4: Truy cập hệ thống

- API Gateway: <http://localhost:8080>
- RabbitMQ Management UI: <http://localhost:15672> (user: guest, pass: guest)

Test 1: Đăng ký người dùng

```
curl -X POST http://localhost:8080/users/register \
```

```
-H "Content-Type: application/json" \
```

```
-d '{
```

```
  "fullName": "Nguyễn Văn A",
```

```
  "email": "nguyenvana@gmail.com",
```

```
  "password": "password123",
```

```
  "phone": "0912345678"
```

```
}'
```

Kết quả:

```
b22dcen0390@MSI:/mnt/f/Project/hospital_appointment_system$ curl -X POST http://localhost:8080/users/register \
-H "Content-Type: application/json" \
-d '{
  "fullName": "Nguyễn Văn A",
  "email": "nguyenvana@gmail.com",
  "password": "password123",
  "phone": "0912345678"
}'
{"message": "User registered successfully", "data": {"id": 4, "full_name": "Nguyễn Văn A", "email": "nguyenvana@gmail.com", "phone": "0912345678", "role": "PATIENT", "created_at": "2026-05-27T11:30:21"}}
```

Test 2: Đăng nhập

```
curl -X POST http://localhost:8080/users/login \
```

```
-H "Content-Type: application/json" \
```

$$-d \quad ' \quad \{$$

```
"email": "nguyenvana@gmail.com",
```

```
"password": "password123"
```

} '

Kết quả

```
b22dcnc039@MSI:/mnt/f/Project/hospital_appointment_system$ curl -X POST http://localhost:8080/users/login \
-H "Content-Type: application/json" \
-d '{
  "email": "nguyenvana@gmail.com",
  "password": "password123"
}'
{"message": "Login successful", "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6NCwiZW1haWwiOiJ1ZjZ3V5ZW52YW5hQGdtYWlsLmNvbnR5cCI6InR5bGVuI0IjQVVRJRU5UIiwiaWF0IjoxNzc5ODgxNjM5LCJleHAiOiJE3NzksNjgwMzI9.kjV05mMyH6hRvkgOE72kF04tCCGw4jjlQUtzEVyVrg4", "data": {"id": 4, "fullName": "Nguyễn Văn A", "email": "nguyenvana@gmail.com", "phone": "0912345678", "role": "PATIENT"}}b22dcnc039@MSI:/mnt
```

Test 3: Lấy danh sách bác sĩ

```
curl -X GET http://localhost:8080/doctors/
```

Kết quả:

```
b22dccn039@MSI:/mnt/f/Project/hospital_appointment_system$ curl -X GET http://localhost:8080/doctors/
{"message":"Doctors retrieved successfully","data":[{"id":1,"name":"Dr. Nguyen Van An","specialization":"Cardiology","phone":"0901000001","email":"an.cardio@hospital.com","room":"A101","description":"Bác sĩ chuyên khoa tim mạch","created_at":"2026-05-26T10:04:12.515Z"}, {"id":2,"name":"Dr. Tran Thi Binh","specialization":"Dermatology","phone":"0901000002","email":"binh.derma@hospital.com","room":"A102","description":"Bác sĩ chuyên khoa da liễu","created_at":"2026-05-26T10:04:12.515Z"}, {"id":3,"name":"Dr. Le Minh Cuong","specialization":"Neurology","phone":"0901000003","email":"cuong.neuro@hospital.com","room":"A103","description":"Bác sĩ chuyên khoa thần kinh","created_at":"2026-05-26T10:04:12.515Z"}, {"id":4,"name":"Dr. Pham Thu Dung","specialization":"Pediatrics","phone":"0901000004","email":"dung.pedia@hospital.com","room":"A104","description":"Bác sĩ chuyên khoa nhi","created_at":"2026-05-26T10:04:12.515Z"}, {"id":5,"name":"Dr. Hoang Quoc Huy","specialization":"Orthopedics","phone":"0901000005","email":"huy.ortho@hospital.com","room":"A105","description":"Bác sĩ chuyên khoa cơ xương khớp","created_at":"2026-05-26T10:04:12.515Z"}]}
```

Test 4: Lấy chi tiết bác sĩ

```
curl -X GET http://localhost:8080/doctors/1
```

Kết quả:

```
b22dccn039@MSI:/mnt/f/Project/hospital_appointment_system$ curl -X GET http://localhost:8080/doctors/1
{"message":"Doctor retrieved successfully","data":{"id":1,"name":"Dr. Nguyen Van An","specialization":"Cardiology","phone":"0901000001","email":"an.cardio@hospital.com","room":"A101","description":"Bác sĩ chuyên khoa tim mạch","created_at":"2026-05-26T10:04:12.515Z"}}b22dccn039@
```

Test 5: Đặt lịch khám

```
curl -X POST http://localhost:8080/appointments/ \
```

```
-H "Content-Type: application/json" \
```

```
-d '{
```

```
  "patientId": 1,
```

```
  "doctorId": 1,
```

```
  "appointmentDate": "2026-05-30",
```

```
  "appointmentTime": "09:00",
```

```
"reason": "Kiểm tra sức khỏe định kỳ"
```

```
}'
```

Kết quả:

```
b22dcn039@MSI:/mnt/f/Project/hospital_appointment_system$ curl -X POST http://localhost:8080/
appointments/ \
-H "Content-Type: application/json" \
-d '{
  "patientId": 1,
  "doctorId": 1,
  "appointmentDate": "2026-05-30",
  "appointmentTime": "09:00",
  "reason": "Kiểm tra sức khỏe định kỳ"
}'
{"message":"Appointment created successfully","doctor":{"id":1,"name":"Dr. Nguyen Van An","sp
ecialization":"Cardiology","phone":"0901000001","email":"an.cardio@hospital.com","room":"A101
","description":"Bác sĩ chuyên khoa tim mạch","created_at":"2026-05-26T10:04:12.515Z"},"data"
:{"id":4,"patient_id":1,"doctor_id":1,"appointment_date":"2026-05-30T00:00:00.000Z","appointm
ent_time":"09:00:00","reason":"Kiểm tra sức khỏe định kỳ","status":"CONFIRMED","created_at":"
```

Test 6: Lấy lịch khám của bệnh nhân

```
curl -X GET http://localhost:8080/appointments/patient/1
```

Kết quả:

```
b22dcn039@MSI:/mnt/f/Project/hospital_appointment_system$ curl -X GET http://localhost:8080/
appointments/patient/1
{"message":"Patient appointments retrieved successfully","data":[{"id":4,"patient_id":1,"doct
or_id":1,"appointment_date":"2026-05-30T00:00:00.000Z","appointment_time":"09:00:00","reason"
:"Kiểm tra sức khỏe định kỳ","status":"CONFIRMED","created_at":"2026-05-27T11:41:24.516Z"}]}
```

Hệ thống được thử nghiệm thông qua một chuỗi các kịch bản tương tác người dùng nhằm kiểm chứng tính đúng đắn của các giao tiếp đồng bộ và bất đồng bộ. Cụ thể, các bước thử nghiệm được tiến hành như sau:

- Người dùng mới tiến hành đăng ký và đăng nhập thành công. Cổng giao tiếp API Gateway định tuyến chính xác yêu cầu đến dịch vụ người dùng và hệ thống trả về mã thông báo xác thực JSON Web Token hợp lệ.
- Người dùng gửi yêu cầu lấy danh sách chuyên khoa. Cổng giao tiếp điều hướng yêu cầu tới dịch vụ bác sĩ và trả về danh sách năm bác sĩ mẫu tương ứng với các chuyên khoa đã được khởi tạo sẵn trong cơ sở dữ liệu.

- Người dùng gửi yêu cầu đặt lịch khám. Hệ thống ghi nhận lịch khám mới được tạo với trạng thái xác nhận. Ngay lập tức, nhật ký hệ thống tại dịch vụ đặt lịch ghi nhận đã phát hành thành công một thông điệp lên kênh sự kiện RabbitMQ. Chuyển sang quan sát tại giao diện quản trị RabbitMQ ở cổng 15672, thông điệp sự kiện xuất hiện trong hàng đợi và ngay lập tức được dịch vụ thông báo tiêu thụ để thực thi tiến trình chạy nền.
- Các thao tác tra cứu lại lịch khám của bệnh nhân hay thao tác cập nhật trạng thái lịch khám thành hủy bỏ đều phản ánh chính xác sự thay đổi dữ liệu trong kho lưu trữ phân tán.

6.3. Đánh giá ưu điểm kiến trúc

- Các vi dịch vụ hoạt động hoàn toàn độc lập, cho phép người quản trị dễ dàng mở rộng theo chiều ngang cho từng dịch vụ cụ thể khi tải trọng tăng cao.
- Việc sử dụng hàng đợi thông điệp để tách rời tiến trình gửi thông báo ra khỏi luồng đặt lịch chính giúp triệt tiêu hiện tượng phong tỏa tiến trình. Cơ chế này đảm bảo thời gian đáp ứng tức thời cho ứng dụng máy khách và tăng cường thông lượng tổng thể của hệ thống.
- Mỗi vi dịch vụ quản lý một cơ sở dữ liệu riêng biệt. Nếu một dịch vụ gặp sự cố, các dịch vụ khác vẫn có thể tiếp tục hoạt động, qua đó loại bỏ hoàn toàn điểm lỗi cục bộ mang tính tập trung.
- Kiến trúc hệ thống cho phép dễ dàng tích hợp thêm các vi dịch vụ mới trong tương lai như dịch vụ thanh toán hay đánh giá chất lượng mà không làm phá vỡ cấu trúc hiện tại.

6.4. Hạn chế và hướng phát triển

Mặc dù đã đáp ứng tốt các yêu cầu cơ bản của bài toán, hệ thống hiện tại vẫn còn một số điểm cần tiếp tục nghiên cứu và hoàn thiện để có thể triển khai trong môi trường thực tế:

- Hệ thống hiện chưa cài đặt cơ chế xác thực và phân quyền cho các giao tiếp nội bộ giữa các vi dịch vụ. Việc bổ sung các kênh truyền bảo mật là cần thiết để chống lại các rủi ro giả mạo dịch vụ.

- Để giảm thiểu độ trễ mạng và giảm tải cho cơ sở dữ liệu quan hệ, hệ thống cần tích hợp thêm cơ chế lưu trữ bộ nhớ đệm tốc độ cao như Redis cho các dữ liệu ít thay đổi.
- Hệ thống hiện thiếu cơ chế giới hạn tần suất yêu cầu truy cập. Cần bổ sung tính năng này tại cổng API Gateway nhằm ngăn chặn nguy cơ quá tải do các cuộc tấn công từ chối dịch vụ gây ra.
- Việc theo dõi luồng thông điệp qua nhiều tiến trình phân tán hiện tại vẫn dựa vào các tệp nhật ký đơn giản. Cần ứng dụng các công cụ giám sát đo lường chuyên sâu như Prometheus và Grafana để có cái nhìn toàn cảnh về tình trạng sức khỏe của hệ thống.

TÀI LIỆU THAM KHẢO

1. Phạm Văn Cường và Nguyễn Xuân Anh, *Giáo trình Các hệ thống phân tán*, Học viện Công nghệ Bưu chính Viễn thông, Khoa Công nghệ thông tin 1, Hà Nội, xuất bản tháng 10 năm 2024.
2. Maarten van Steen và Andrew S. Tanenbaum, *Distributed Systems*, Ấn bản lần thứ 3, Phiên bản 3.03, xuất bản năm 2020.
3. George Coulouris, Jean Dollimore, Tim Kindberg và Gordon Blair, *Distributed systems: Concepts and Design*, Ấn bản lần thứ 5, nhà xuất bản Addison-Wesley, năm 2012.